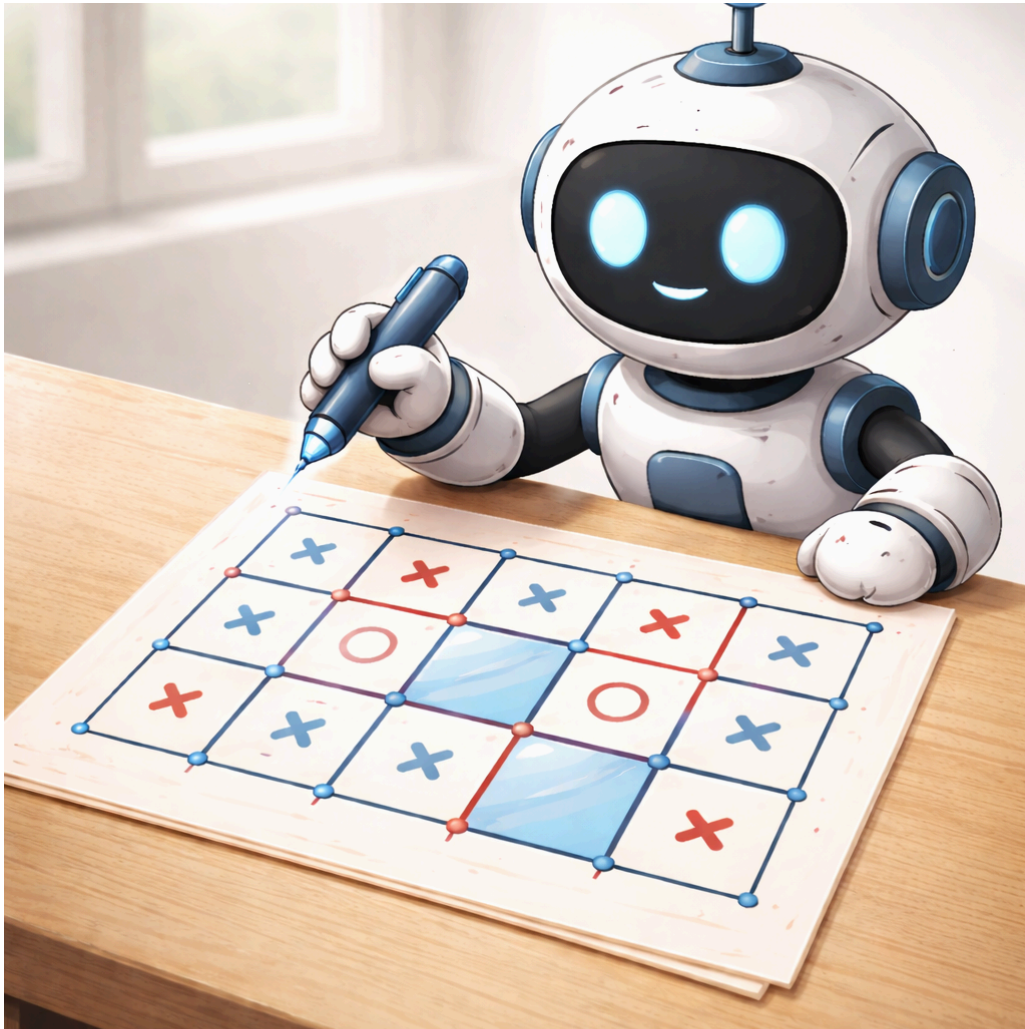


Rapport de Projet

Pipopipette



Jeremy Rakotonirina, Eduardo Cobos Fernandez

Partie 1

Question1

Comment est représenté l'état du jeu ?

L'état du jeu est représenté par la classe **Board**. Elle contient l'ensemble des segments tracés (stockés dans *hEdges* et *vEdges*) et les cases capturées par chaque joueur (boxes). On peut extraire ces informations avec *getHEdges()*, *getVEdges()* et *getBoxOwner(int r, int c)*. De plus, la classe Board peut nous dire les coups possibles à jouer, si la partie est finie et le score courant de chaque joueur avec *getAvailableActions()*, *isFinished()* et *getScore(int playerId)* respectivement. Ayant tous ces informations avec la classe Board, elle correspond à la représentation de l'état du jeu.

Comment sont modélisées les actions ?

Les actions sont modélisées grâce à la classe **Action**. Étant la seule action possible de tracer un segment, chaque action sera définie par sa position (ligne et colonne) codée dans *row* et *col* ainsi que par son orientation donnée par le *type* de l'action qui va dire si le segment est horizontal ou vertical. On pourra extraire toutes les actions possibles à un instant *t* en appliquant *getAvailableActions()* au Board représentant l'état du jeu à l'instant *t*.

Comment est implémentée la fonction de transition ?

La fonction de transition $T(s,a)$ qui applique l'action *a* à l'état *s* est implémentée par la méthode *Board.apply(Action action, int playerId)*. La méthode trace le segment décrit par l'action *a* et retourne le nombre de cases fermées par cette action en attribuant les cases au joueur qui les ferme.

Comment est calculé le score ?

Le score est par la méthode *Board.getScore(int playerId)* qui va donner le score de l'état du jeu représenté par Board (elle parcourt le tableau *boxes[][]* et calcule toutes les cases occupées par le joueur d'ID *playerID*).

Comment est géré le fait qu'un joueur puisse rejouer ?

Le fait qu'un joueur puisse rejouer est gérée dans la boucle principale, *DotsBoxesGame.play()*. Par défaut, le joueur garde le tour et peut rejouer mais lorsque après son action il n'a fermé aucune case le tour est donnée à l'autre joueur avec *switchPlayer()*.

Question 2

Donnez la définition formelle du jeu Pipopipette.

Joueurs $P = \{0, 1\}$; on a deux joueurs qui s'affrontent et qui sont identifiés par leur identifiant 0 ou 1.

Ensemble des états S = tous les objets Board possibles. Ou plus simplement l'ensemble de tous les triplets ($hEdges$, $vEdges$, $boxes$) possibles où $hEdges$ est une matrice booléenne disant si les segments horizontaux sont tracés ou non, $vEdges$ est une matrice booléenne disant si les segments verticaux sont tracés ou non et $boxes$ est une matrice d'entiers qui va donner le propriétaire de chaque case. -1 si elle n'a pas été fermée et le Id du joueur qui l'a fermé si elle a été fermée.

Ensemble des actions A = tous les objets Action possibles. C'est à dire tous les triplets ($type$, row , col) représentant le traçage d'un segment où col et row sont la colonne et la ligne dans lesquelles le segment va être tracé et $type$ est l'orientation du segment, c'est à dire une variable disant horizontal ou vertical.

Fonction de transition $T : T(s, a, p) \rightarrow (s', p')$. T va prendre en argument un triplet (s, a, p) où s est l'objet Board représentant l'état du jeu à ce moment, a l'objet Action représentant l'action que le joueur a effectué et p l'ID du joueur qui a effectué l'action a . La fonction va renvoyer le couple (s', p') où s' est un objet Board représentant le nouveau état du jeu après l'action a et p' est l'ID du joueur à qui c'est le tour de jouer. Si après l'action a aucune case est fermée alors $p' = p + 1 \ [2]$; sinon $p' = p$.

Fonction d'Utilité $U : U(s, p) = s.getScore(p)$. Où s est l'objet Board représentant l'état du jeu à ce moment et p est le joueur pour lequel on veut savoir le score. Lorsque l'état terminal est atteint ($s.isFinished() == true$) le gagnant est donné par $\max(s.getScore(0), s.getScore(1))$.

Question 3

Quel est le facteur de branchement au début de la partie pour une grille $N \times M$?

Le facteur de branchement initial est de $2 \times N \times M - N - M$ car ça correspond au nombre total de segments disponibles au début de la partie.

Comment évolue-t-il au cours de la partie ?

Au cours de la partie le facteur de branchement va être réduit de 1 après chaque action. Car chaque fois que l'on trace un segment on utilise une action possible.

Quelle est la profondeur maximale de l'arbre de jeu ?

La profondeur maximale de l'arbre de jeu est de $2^N \cdot M - N - M$. Car au début de la partie il y a ce nombre de coups à jouer et chaque action réduit le nombre de coups par 1 donc si la partie fini quand il n'y a plus de coups à jouer la profondeur de l'arbre sera de $2^N \cdot M - N - M$.

Le jeu est-il strictement alterné ? Quelles conséquences cela a-t-il sur Minimax ?

Non, le jeu n'est pas strictement alterné, si un joueur ferme une case il rejoue. On ne peut donc plus juste alterner les niveaux dans MinMax à chaque profondeur. À chaque niveau il faudra déterminer dynamiquement selon le résultat du coup précédent si on est dans un niveau min ou dans un niveau max. Il faudra passer le joueur courant en paramètre récursif plutôt que de déduire le niveau MinMax à partir de la profondeur.

Question 4

Après avoir joué plusieurs parties on peut identifier les stratégies clés du jeu :

Fermeture de cases :

Lorsque l'on a une case à trois cotés il suffit de tracer le côté manquant pour gagner la case et ensuite on peut rejouer. On constate que l'idée ne va pas vraiment être de fermer des cases mais d'éviter de donner des cases de trois côtés à l'adversaire car on lui donne des tours et des points gratuitement.

Chaînes de cases :

Vers la fin de la partie on se rend compte que les chaînes rapportent beaucoup de points et peuvent complètement changer le cours d'une partie. On a une chaîne lorsque plusieurs cases à 2 côtés sont adjacentes et un joueur transforme l'une d'entre elles en une case de 3 côtés ; alors, le rival pourra réclamer la chaîne entière pour lui facilement et emporter beaucoup de points. Il faudra toujours éviter d'ouvrir des chaînes longues tout en forçant le rival à les ouvrir pour nous.

Cases de 2 côtés :

Il y a une particularité observée lorsque l'on joue entre humains. Étant susceptibles de faire des erreurs, créer des cases de 2 côtés crée une fenêtre pour que l'adversaire la transforme en une case à trois côtés par erreur, nous faisant gagner un point. Ce n'est pas vraiment une stratégie mais pourra être utilisé pour la création d'heuristiques, notamment en vue d'une partie IA contre Humain.

Stratégies que l'IA devra intégrer :

On peut en sortir trois comportements que toute bonne IA devra avoir :

- Éviter de créer des cases à trois côtés inutilement.
- Éviter d'ouvrir des chaînes tant que c'est possible.
- Si donner des cases à l'autre joueur devient inévitable, il faut que l'IA fasse en sorte de lui donner le moins de cases possibles. Prioriser les cases individuelles aux chaînes et prioriser les chaînes courtes aux longues.

Partie 2

Implémentation automate Random

On a implémenté l'automate qui va jouer aléatoirement. Voici comment il procède :

- Récupère la liste des actions disponibles
- Si aucune action n'est disponible renvoi null
- Sinon renvoi une action sélectionné aléatoirement

Implémentation automate glouton

On a implémenté l'automate qui va jouer l'action qui va compléter le plus grand nombre de cases, s'il en a pas il va jouer aléatoirement. Voici comment il procède :

- Récupère la liste des actions disponibles
- Si aucune action n'est disponible renvoi null
- Sinon va chercher action par action laquelle complète le maximum de cases (s'il en trouve une qui complète 2 il s'arrête car 2 cases est le maximum que l'on peut compléter en une action)
- Si aucune action ne ferme de case il joue une action aléatoire. La classe *GloutonActionStrategy* accepte de façon optionnelle une graine (seed) afin de rendre cet aléatoire déterministe en vue des tests notamment.

Idée : Implémenter une heuristique simple pour ne pas faire des choix aux hasard mais réfléchir un minimum (ex : ne pas laisser des cases faciles à compléter pour l'adversaire)

Comparaison des automates

On va procéder à l'analyse des deux automates. Pour ceci on a implémenté la classe Comparaison qui va nous permettre de simuler autant de parties que l'on veut entre deux automates en n'affichant que les statistiques de victoire. Elle prend en paramètres la taille du plateau, le nombre de parties à faire et les stratégies à comparer. On va alterner les rôles de chaque stratégie à chaque partie pour être équitable et que chaque stratégie commence autant de parties que l'autre. À la fin de la simulation on aura le nombre de victoires de chaque stratégie, le nombre d'égalités et la marge moyenne de victoire. De plus, un timeout par coup est appliqué pour éviter qu'une stratégie trop lente ne bloque l'exécution. Pour l'analyse des IAs d'autres classes ont été créés.

Après le lancement de simulations avec plusieurs plateaux différents avec le command :
*mvn compile exec:java -Dexec.mainClass="DotsBoxes.Comparaison" -Dexec.args="n m 100
glouton random"* on peut dresser le tableau suivant :

Plateau	Victoires glouton	Victoires random	Égalités	Marge moyenne
3x3	77%	4%	19%	+2,46 cases
4x4	99%	1%	0%	+6,80 cases
5x5	100%	0%	0%	+13,82 cases
6x6	100%	0%	0%	+22,76 cases

On peut en tirer que le glouton domine largement le random et que plus la taille du plateau augmente plus cet écart augmente. Non seulement le pourcentage de victoire augmente mais l'écart moyen de cases gagnés explose. C'est logique car le random ne voit pas s' il y a des coups à jouer qui ferment des cases ou pas.

En 3x3 en revanche, on observe encore 19% d'égalités et 4% de victoires pour le random. Cela s'explique par la petite taille du plateau : il y a moins de cases à fermer, donc moins d'occasions d'exploiter l'avantage glouton, et la part du hasard est plus importante.

Cependant on peut observer les limites de l'algorithme glouton : lorsque aucune case ne peut être fermée il joue au hasard, pouvant créer des cases à 3 côtés ou directement offrir des chaînes entières à l'adversaire. Face à un adversaire naïf comme le random, cette faiblesse ne se voit pas ; mais il suffit de jouer quelques parties contre lui pour la remarquer.

C'est là qu'intervient Minimax : plutôt que de jouer au hasard quand aucune case n'est fermable, il explore les coups futurs et choisit celui qui minimise ce qu'il offre à l'adversaire. Il transforme ainsi le point faible du glouton en une décision raisonnée.

Partie 3

Implémentation MinMax

On a implémenté l'algorithme Minimax avec une profondeur limite. On a considéré que le joueur 0 est le maximiseur et le joueur 1 le minimiseur. Et on a pris en compte le fait que le joueur qui ferme une case rejoue. Voici comment il procède :

- Récupère la liste des actions disponibles
- Si aucune action n'est disponible renvoi null

- Sinon lance récursivement *minmax(board, profondeur, isMaxPlayer)* :
 - Si on est sur une feuille, la partie est terminée ou la profondeur maximale de la recherche est atteinte. On renvoie le score de la partie au moment. C'est une implémentation sans heuristique qui ne cherche pas à savoir ce qui va se passer après la feuille.
 - Sinon, on regarde toutes les actions possibles en simulant leur effet sur une copie du plateau. Le maximiseur choisit l'action qui mène au meilleur score pour lui, le minimiseur celle qui mène au pire score pour son adversaire. Pour respecter la règle de rejouer, si une action ferme une case alors le prochain nœud est du même type (max ou min), sinon il est de type opposé.

Pour vérifier la bonne implémentation de notre algorithme on a différents types de tests:

Tests de robustesse : on vérifie que `selectAction` renvoie null quand aucune action n'est disponible, qu'elle renvoie une action valide sur un plateau vide, qu'elle reste valide tout au long d'une partie complète et qu'elle ne modifie pas le plateau original lors de la recherche.

Tests de comportement stratégique : on vérifie que Minimax trouve bien le coup optimal dans des situations simples (il peut fermer une case, il préfère fermer deux cases plutôt qu'une) qu'il évite de jouer un coup qui donnerait une case gratuite à l'adversaire, et qu'il préfère donner une case à l'adversaire plutôt que deux quand il n'a pas le choix. De plus, on a testé que la profondeur était respectée et qu'il tombait dans des pièges qui étaient cachés à une profondeur qu'il ne voyait pas.

Idée d'amélioration : on a codé le minmax sans heuristique, quand la profondeur limite est atteinte, l'algorithme n'évalue que les cases déjà fermées. On peut implémenter une heuristique pour mieux estimer la valeur d'un état non final (par exemple pénaliser les chaînes ouvertes offertes à l'adversaire).

Implémentation Heuristique

On a ajouté une heuristique pour mieux estimer la valeur d'un état non final lorsque la profondeur maximale est atteinte. Au lieu de renvoyer uniquement le score courant (qui ignore l'état du plateau et donc les cases qu'il offre à l'adversaire), la fonction `evaluate` enrichit ce score en analysant le plateau. Elle distingue trois situations :

- Une case isolée à 3 côtés (non adjacente à une case à 2 côtés) : le joueur actuel peut la capturer sans risque d'ouvrir une chaîne, elle ajoute donc +1 ou -1 au score selon le joueur.
- Une case isolée à 2 côtés (non liée à une chaîne) : l'adversaire pourrait la transformer en case à 3 côtés par erreur, elle ajoute donc +0.25 indépendamment du joueur.
- Une chaîne (une case à 3 côtés adjacente à des cases à 2 côtés) : l'heuristique en calcule la taille et attribue au joueur actuel des points proportionnels à cette taille, car c'est lui qui s'apprête à la capturer.

On constate son efficacité par le fait que le test où Minimax était censé tomber dans un piège ne passe plus : l'heuristique lui permet d'éviter des pièges qu'il ne pouvait pas voir sans elle. Une analyse expérimentale viendra confirmer ce gain de performance.

Analyse expérimentale de la profondeur de recherche

Pour analyser l'impact de la profondeur de recherche sur les performances et le temps moyen de décision on a calculé les statistiques de Minimax pour plusieurs profondeurs sur des plateaux de taille croissante grâce à la classe *ComparaisonMinimaxAlphaBeta* (elle a été modifiée par la suite pour faire la comparaison avec alphaBeta mais elle revoit encore les statistiques importantes pour cette analyse).

Facteur de branchement

Le facteur de branchement correspond au nombre d'actions disponibles à chaque nœud de l'arbre. Un plateau $n \times m$ possède $2 \cdot n \cdot m + n + m$ arêtes au total, soit 24 pour un 3×3 et 40 pour un 4×4 . À profondeur fixe 3, Minimax explore 1465 nœuds sur un plateau 3×3 contre 12721 sur un 4×4 , soit presque 9 fois plus. Chaque fois que l'on augmente la profondeur le nombre de nœuds est multiplié par ce facteur, ceci entraîne une croissance exponentielle du temps de calcul. Il faudra donc trouver un compromis entre avoir une profondeur qui permette d'avoir des bons résultats et une profondeur qui permette d'avoir un temps de calcul raisonnable.

Compromis profondeur / temps de calcul

Profondeur	Minimax 3×3 (ms)	Minimax 4×4 (ms)
1 – 2	< 1	< 1
3	1	11
4	11	251
5	101	5 211
6	727	102 856

Sur un plateau 3×3 , le temps est multiplié par 7 ou 10 à chaque niveau supplémentaire. Sur un 4×4 , l'explosion est encore plus marquante, la profondeur 6 prend plus de 100 secondes contre 727ms sur un 3×3 . On peut donc conclure que agrandir les plateaux aura aussi un impact énorme sur le temps de calcul.

Qualité du jeu selon la profondeur

Pour vérifier que la profondeur améliore les décisions, on a fait s'affronter Minimax à profondeur 4 contre Minimax à profondeur 2 sur un plateau 3×3. Voici les résultats obtenus :

Profondeur	Victoires	Défaites	Égalités	Marge moyenne
4	10	0	10	+2,00 cases
2	0	10	10	-2,00 cases

La profondeur 4 domine clairement la profondeur 2. En revanche, au-delà d'un certain seuil les résultats se stabilisent : profondeur 3 et profondeur 4 produisent les mêmes décisions sur 3×3, l'algorithme explorant alors suffisamment loin pour voir les mêmes situations.

Profondeur raisonnable selon la taille du plateau

En fixant un budget de 500ms par coup, la profondeur maximale atteignable par Minimax est :

Plateau	Profondeur max (Minimax)
3×3	5
4×4	4
5×5	≤ 3 (estimé)

Ces limites montrent que Minimax atteint rapidement un temps de calcul beaucoup trop élevé dès que le plateau grandit. Pour atteindre des profondeurs plus élevées dans le même temps, il faut un algorithme capable d'élaguer les branches inutiles de l'arbre. D'ici la transition logique vers l'implémentation de l'algorithme Alpha-Beta, qui sera présenté dans la section suivante.

Implémentation Alpha-Beta

Analyse Alpha-Beta vs Minmax

```

=== Nœuds explorés ? plateau 3x3 ? profondeur 3 ===
Coup   MM nœuds   AB nœuds   Coupes ?+?   Réduction
1       1465       265        31           81,9%
2       1112       235        21           78,9%
3        821       201        24           75,5%
4        586       199        28           66,0%
5        401       196        29           51,1%
6        260       146        19           43,8%

=== Nœuds explorés ? plateau 4x4 ? profondeur 3 ===
Coup   MM nœuds   AB nœuds   Coupes ?+?   Réduction
1      12721      1081        69          91,5%
2      11156      1062        74          90,5%
3       9725       957        73          90,2%
4       8422      1457       144          82,7%
5       7241      1402        89          80,6%
6       6176      1236        78          80,0%

=== Temps de calcul ? plateau 3x3 ===
Prof.   Minimax (ms)   Alpha-Beta (ms)   Accélération
1         0           0                 N/A
2         0           0                 N/A
3         2           0                 N/A
4        21           1                x21,0
5       177           3                x59,0
6      1362          17               x80,1

=== Temps de calcul ? plateau 4x4 ===
Prof.   Minimax (ms)   Alpha-Beta (ms)   Accélération
1         0           0                 N/A
2         1           0                 N/A
3        25           2                x12,5
4       546           7                x78,0
5      11346          70               x162,1
6     224743         341              x659,1
[Minimax trop lent au-delà, arrêt anticipé]

```

Noeuds explorés:

Au niveau des noeuds explorés il est clair que l'Alpha Beta marche bien puisqu'il explore beaucoup moins de noeuds grâce à ses coupes. Plus on avance dans le jeu, moins il devient efficace ce qui est logique.

Temps de calcul:

Alpha Beta est beaucoup plus rapide que Minimax également. Plus on avance dans le jeu, plus il est rapide grâce à son nombre de nœuds qui est bien inférieur.

```

=== Profondeur maximale sous 500ms ? plateau 3x3 ===
Minimax : profondeur max = 5
Alpha-Beta : profondeur max = 8
? Alpha-Beta atteint 3 niveaux de plus dans le même budget.

=== Profondeur maximale sous 500ms ? plateau 4x4 ===
Minimax : profondeur max = 3
Alpha-Beta : profondeur max = 6
? Alpha-Beta atteint 3 niveaux de plus dans le même budget.

=== Qualité des décisions ? plateau 3x3 (20 parties) ===
MM depth  AB depth  V(MM)  V(AB)  Egal.  Marge moy
.
2          2          0        0      20      +0,00
2          3          10       10       0      -1,00
2          4          0        10      10      -2,00
3          2          10       10       0      +1,00
3          3          10       10       0      +0,00
3          4          10       10       0      +0,00
4          2          10        0      10      +2,00
4          3          10       10       0      +0,00
4          4          10       10       0      +0,00

=== Qualité des décisions ? plateau 4x4 (20 parties) ===
MM depth  AB depth  V(MM)  V(AB)  Egal.  Marge moy
.
2          2          10       10       0      +0,00
2          3          10       10       0      +0,00
2          4          10       10       0      +2,00
3          2          10       10       0      +0,00
3          3          10       10       0      +0,00
3          4          10       10       0      +1,00
4          2          10       10       0      -2,00
4          3          10       10       0      -1,00
4          4          10       10       0      +0,00

```

Analyse terminée.

Profondeur maximale

Alpha Beta va plus loin puisqu'il a moins de noeuds à explorer.

Qualité des décisions

Sur les plateaux 3*3 et 4*4 on voit que la qualité de décision est la même puisqu'il sont quasiment égaux, Alpha Beta est définitivement plus optimal.

Partie 4

Implémentation MCTS

Concernant la décision finale dans l'algorithme, le cours dit de prendre l'action qui a le gain moyen le plus haut (calculé par score/nbvisites). Mais on a modifié en prenant l'action qui a le plus de visites pour que ce soit plus robuste sur le bruit (on rappelle que la simulation est purement aléatoire). On peut avoir des actions qui ont un gain moyen très haut car il a eu peu de visite et une simulation atypique qui a gonflé son score.

Concernant le critère d'exploration pendant la sélection on a gardé le Upper Confidence Bound 1 du cours.

Comparaison MCTS vs AlphaBeta/MinMax seul

On a une approche où on a pris un temps en ms et on a regardé pendant ce temps combien le MCTS faisait d'itérations et la profondeur du MINMAX et ALPHABETA. Puis on a configuré leur algorithme avec ces données et on a regardé sur 30 parties les résultats.

```
=== Budget-temps égal (1000ms/coup) ? plateau 3x3 ===
Calibration : MCTS=102400 iter | Minimax=prof.5 | Alpha-Beta=prof.8
MCTS(102400)          vs Minimax(d=5)          : 9 V / 15 D / 6 N
(30% / 50% / 20%)
MCTS(102400)          vs AlphaBeta(d=8)         : 0 V / 15 D / 15 N
(0% / 50% / 50%)
```

```
=== Budget-temps égal (500ms/coup) ? plateau 4x4 ===
Calibration : MCTS=12800 iter | Minimax=prof.3 | Alpha-Beta=prof.5
MCTS(12800)           vs Minimax(d=3)           : 4 V / 26 D / 0 N
(13% / 87% / 0%)
MCTS(12800)           vs AlphaBeta(d=5)          : 3 V / 27 D / 0 N
(10% / 90% / 0%)
```

Le MiniMax et Alpha Beta sont très largement meilleur que le MCTS.

IA Experte

Stratégie experte — Méthode et justifications théoriques

1. Architecture à deux modes

La stratégie repose sur une observation clé : le coût d'une copie de plateau et d'une recherche arborescente est raisonnable sur une grille avec peu de coups disponibles, mais explose sur de grandes grilles. On distingue donc deux régimes selon le nombre de segments restants :

- Mode Alpha-Beta (≤ 80 coups disponibles) : recherche arborescente avec approfondissement itératif

- Mode glouton chaîné (> 80 coups) : décision en $O(N)$ sans copie de plateau

Le seuil de 80 correspond à la frontière entre une grille 5×6 (71 segments) et une 6×6 (84 segments), au-delà de laquelle les copies de plateau rendent l'arbre de recherche trop coûteux dans le budget de 850 ms.

2. Théorie des chaînes

Dots and Boxes possède une structure connue : une chaîne est un ensemble connexe de cases ayant exactement 2 côtés tracés, reliées par des segments non tracés. Dès qu'un joueur « ouvre » une chaîne (donne le 3e côté à une case), son adversaire peut capturer l'intégralité de la chaîne en cascade.

Cette propriété induit une hiérarchie de décision, que nous avons décidée d'appliquer:

- 1) Capturer si une case à 3 côtés existe (gain immédiat garanti)
- 2) Jouer un coup sûr (aucun 3e côté offert à l'adversaire)
- 3) Si contraint, ouvrir la chaîne la plus courte (sacrifice minimal)

3. Mode glouton — grandes grilles

Sur les grandes grilles, l'enjeu principal est la détection rapide de chaînes sans copie de plateau. On a codé 3 primitives $O(1)$:

captureCount(a) : nombre de cases fermées par l'action (vérifie si les cases adjacentes ont déjà 3 côtés)

isSafe(a) : vrai si l'action ne donne le 3e côté à aucune case adjacente

safePriority(a) : parmi les coups sûrs, préférence à ceux qui donnent le moins de côtés aux cases voisines (minimise la création de futures chaînes)

Quand aucun coup sûr n'existe, la taille de chaque chaîne ouvrable est calculée par un BFS depuis les cases adjacentes à l'action. On choisit le coup qui ouvre la chaîne la plus courte.

4. Mode Alpha-Beta — petites et moyennes grilles

Approfondissement itératif

La recherche Alpha-Beta est lancée avec des profondeurs croissantes (1, 2, 3, ...) jusqu'à épuisement du budget de 850 ms. À chaque profondeur, si la recherche se termine dans le temps imparti, le meilleur coup est mis à jour. Cette approche garantit qu'un coup valide est toujours disponible même en cas de timeout, et que la profondeur atteinte est maximale pour le temps donné.

Ordonnancement des coups

L'efficacité d'Alpha-Beta dépend fortement de l'ordre d'exploration. Deux niveaux sont utilisés :

Grilles ≤ 30 coups (3×3, 4×4) : ordonnancement simple — +1000 pour une capture, -200 pour un coup dangereux. Sur ces grilles, appeler le BFS de chaîne à chaque nœud ralentirait trop la recherche et réduirait la profondeur atteignable.

Grilles > 30 coups : ordonnancement avec taille de chaîne — la pénalité pour un coup dangereux est proportionnelle à $200 \times \text{taille_chaîne_ouverte}$. Les coups qui ouvrent de longues chaînes sont ainsi repoussés en fin de liste, améliorant l'élagage.

Fonction d'évaluation

L'heuristique calcule $\text{score}(\text{joueur } 0) - \text{score}(\text{joueur } 1)$ et y ajoute une estimation des gains futurs :

Cases à 3 côtés : le joueur courant les capturera au prochain coup. On identifie l'unique côté non tracé de chaque case, et on ajoute la taille de la chaîne qui peut être atteinte depuis ce voisin.

Cases à 2 côtés isolées : bonus mineur de 0,25 représentant leur potentiel (elles deviendront capturables si leur chaîne est ouverte).

5. Gestion du temps

Le budget de 850 ms par coup laisse une marge de 150 ms pour les frais généraux (rendu graphique, copie initiale de la liste d'actions). La vérification `isTimeUp()` est effectuée à chaque nœud de l'arbre Alpha-Beta et à chaque itération des boucles du mode glouton.

Comparaison IA experte - AlphaBeta

```
Plateau 3x3 (24 segments) ? AB profondeur 6
  Expert : 5 V / 5 D / 0 N (50%) | temps moy: 376 ms
  AlphaBeta: 5 V / 5 D / 0 N (50%) | temps moy: 309 ms
  Score moyen Expert : +1,0 cases/partie
  ? Dépassements >1s détectés : 10 fois

Plateau 4x4 (40 segments) ? AB profondeur 5
  Expert : 10 V / 0 D / 0 N (100%) | temps moy: 418 ms
  AlphaBeta: 0 V / 10 D / 0 N (0%) | temps moy: 933 ms
  Score moyen Expert : +5,0 cases/partie
  ? Dépassements >1s détectés : 10 fois

Plateau 5x5 (60 segments) ? AB profondeur 4
  Expert : 5 V / 5 D / 0 N (50%) | temps moy: 523 ms
  AlphaBeta: 5 V / 5 D / 0 N (50%) | temps moy: 455 ms
  Score moyen Expert : -2,0 cases/partie
  ? Dépassements >1s détectés : 10 fois

Plateau 6x6 (84 segments) ? AB profondeur 3
  Expert : 10 V / 0 D / 0 N (100%) | temps moy: 393 ms
  AlphaBeta: 0 V / 10 D / 0 N (0%) | temps moy: 166 ms
  Score moyen Expert : +24,0 cases/partie

Plateau 7x7 (112 segments) ? AB profondeur 3
  Expert : 5 V / 5 D / 0 N (50%) | temps moy: 243 ms
  AlphaBeta: 5 V / 5 D / 0 N (50%) | temps moy: 276 ms
  Score moyen Expert : +6,0 cases/partie
```

On voit que dans la plupart des cas l'IA Experte est plus forte que le Alpha Beta. Pour le 3*3 ils sont à égalité et c'est normal car le plateau est petit. Pour le plateau 5*5 et 7*7, nous n'avons pas bien compris pourquoi ils étaient à égalité mais après une longue réflexion et des tests pour essayer de résoudre ce problème, nous avons trouvé que c'était en raison de la structure des chaînes qui émerge naturellement sur ces grilles lors d'une partie entre deux joueurs appliquant la même stratégie. Sur les grilles carrées impaires comme 5×5, 7×), la symétrie du plateau tend à produire un nombre impair de chaînes longues en fin de partie, ce qui défavorise systématiquement le même joueur selon l'ordre d'ouverture et de plus nous n'avons pas codé le double crossing qui permet de contrer cette règle.